

Experiment No. 03

UPI Payment and Digital Wallet System using Interface and Exception Handling

Date: 17/09/2026

■ Objective / Aim

To design and implement a **UPI Payment and Digital Wallet System** in Java using **interfaces and exception handling**, simulating real-world digital payment operations such as wallet creation, money addition, UPI payments, balance checking, and transaction validation.

■■ Problem Statement

Design a **Digital Wallet System** for a user. The system should allow a user to:

- Create a wallet with a name, mobile number, and UPI ID.
- Add money to the wallet.
- Make a payment using the UPI ID of another user.
- Check the available wallet balance.
- Validate the UPI ID before processing a payment.
- Prevent payments when the balance is insufficient.
- Prevent invalid or negative transaction amounts.
- Handle invalid transactions using appropriate exceptions.
- Display the final wallet details and transaction status.

■ Concepts Demonstrated

- Interface
- Encapsulation
- Classes and Objects
- Constructors
- Method Implementation
- String Operations
- Custom Exceptions
- try-catch-finally
- Exception Handling
- Validation

■ Class and Interface Design

1. *PaymentService Interface*

Defines the common operations: **pay()** and **checkBalance()**.

2. *Wallet Class*

Stores user name, mobile number, UPI ID, and balance. The data members are declared **private** to demonstrate encapsulation. Methods include **addMoney()**, **getBalance()**, and **displayWalletDetails()**.

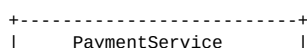
3. *UPIPayment Class*

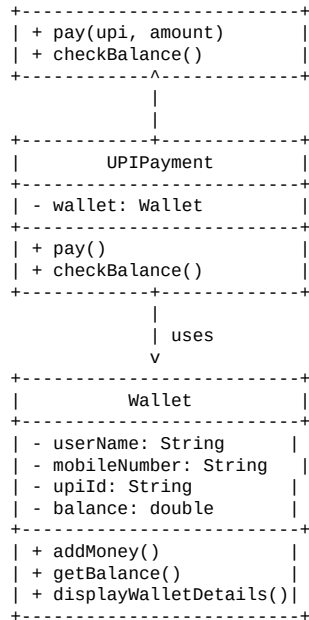
Implements the **PaymentService** interface and provides **pay()** and **checkBalance()**.

4. *Custom Exceptions*

InsufficientBalanceException, **InvalidUPIException**, and **InvalidAmountException** are created to handle invalid transaction conditions.

■ UML Class Diagram





Custom Exceptions:
 InsufficientBalanceException
 InvalidUPIException
 InvalidAmountException

■ Algorithm / Steps

1. Create an interface named **PaymentService**.
2. Declare pay() and checkBalance() methods in the interface.
3. Create a Wallet class.
4. Declare user information and balance as private data members.
5. Create a constructor to initialize the wallet details.
6. Define addMoney() to add money to the wallet.
7. Create an UPIPayment class that implements PaymentService.
8. Create a Wallet object and associate it with the UPIPayment object.
9. Validate the UPI ID using String operations.
10. If the UPI ID is invalid, throw InvalidUPIException.
11. Validate the payment amount.
12. If the amount is zero or negative, throw InvalidAmountException.
13. Check whether sufficient balance is available.
14. If the balance is insufficient, throw InsufficientBalanceException.
15. If all validations are successful, deduct the payment amount from the wallet balance.
16. Display the payment success message.
17. Use try-catch-finally to handle transaction exceptions.
18. Display the final wallet balance.

■ Source Code

```

interface PaymentService {
    void pay(String upiId, double amount)
        throws InvalidUPIException,
               InvalidAmountException,
               InsufficientBalanceException;

    void checkBalance();
}

class Wallet {
    private String userName;
    private String mobileNumber;
    private String upiId;
    private double balance;

    Wallet(String userName, String mobileNumber,
           String upiId) {
        this.userName = userName;
        this.mobileNumber = mobileNumber;
        this.upiId = upiId;
        this.balance = 0.0;
    }
}

```

```

    public void addMoney(double amount)
        throws InvalidAmountException {
        if (amount <= 0) {
            throw new InvalidAmountException(
                "Amount must be greater than zero.");
        }

        balance += amount;
        System.out.println("Money added: Rs." + amount);
    }

    public double getBalance() {
        return balance;
    }

    public String getUpiId() {
        return upiId;
    }

    public void displayWalletDetails() {
        System.out.println("\n===== WALLET DETAILS =====");
        System.out.println("User Name      : " + userName);
        System.out.println("Mobile Number : " + mobileNumber);
        System.out.println("UPI ID       : " + upiId);
        System.out.printf("Balance      : Rs.%.2f%n", balance);
    }

    public void deductMoney(double amount) {
        balance -= amount;
    }
}

class InsufficientBalanceException extends Exception {
    InsufficientBalanceException(String message) {
        super(message);
    }
}

class InvalidUPIException extends Exception {
    InvalidUPIException(String message) {
        super(message);
    }
}

class InvalidAmountException extends Exception {
    InvalidAmountException(String message) {
        super(message);
    }
}

class UPIPayment implements PaymentService {
    private Wallet wallet;

    UPIPayment(Wallet wallet) {
        this.wallet = wallet;
    }

    @Override
    public void pay(String upiId, double amount)
        throws InvalidUPIException,
               InvalidAmountException,
               InsufficientBalanceException {

        // UPI validation using String operations
        if (upiId == null || !upiId.contains("@")) {
            throw new InvalidUPIException(
                "Invalid UPI ID.");
        }

        // Amount validation
        if (amount <= 0) {
            throw new InvalidAmountException(
                "Payment amount must be greater than zero.");
        }

        // Balance validation
        if (amount > wallet.getBalance()) {
            throw new InsufficientBalanceException(
                "Insufficient wallet balance.");
        }

        wallet.deductMoney(amount);

        System.out.println("Payment Successful!");
        System.out.println("Paid To      : " + upiId);
        System.out.println("Amount       : Rs." + amount);
        System.out.println("Transaction completed successfully.");
    }

    @Override
    public void checkBalance() {
        System.out.printf("Available Balance: Rs.%.2f%n",

```

```

        wallet.getBalance());
    }
}

public class UPIDigitalWallet {
    public static void main(String[] args) {

        Wallet wallet = new Wallet(
            "Rahul",
            "9876543210",
            "rahul@upi");

        UPIPayment payment = new UPIPayment(wallet);

        try {
            wallet.addMoney(10000);
        } catch (InvalidAmountException e) {
            System.out.println("Error: " + e.getMessage());
        }

        wallet.displayWalletDetails();

        System.out.println("\n===== UPI PAYMENT =====");

        try {
            payment.pay("priya@upi", 2500);
        } catch (InvalidUPIException |
            InvalidAmountException |
            InsufficientBalanceException e) {
            System.out.println("Transaction Failed: "
                + e.getMessage());
        } finally {
            System.out.println("Transaction process completed.");
        }

        System.out.println("\n===== BALANCE CHECK =====");
        payment.checkBalance();

        System.out.println("\n===== INVALID TRANSACTION TEST =====");

        try {
            payment.pay("invalidupi", 2000);
        } catch (InvalidUPIException |
            InvalidAmountException |
            InsufficientBalanceException e) {
            System.out.println("Transaction Failed: "
                + e.getMessage());
        } finally {
            System.out.println("Transaction process completed.");
        }

        System.out.println("\n===== FINAL WALLET DETAILS =====");
        wallet.displayWalletDetails();
    }
}

```

■ Compilation and Execution

File name: UPIDigitalWallet.java

```

javac UPIDigitalWallet.java
java UPIDigitalWallet

```

■■ Sample Output

```

Money added: Rs.10000.0

===== WALLET DETAILS =====
User Name      : Rahul
Mobile Number  : 9876543210
UPI ID         : rahul@upi
Balance       : Rs.10000.00

===== UPI PAYMENT =====
Payment Successful!
Paid To        : priya@upi
Amount         : Rs.2500.0
Transaction completed successfully.
Transaction process completed.

===== BALANCE CHECK =====
Available Balance: Rs.7500.00

===== INVALID TRANSACTION TEST =====
Transaction Failed: Invalid UPI ID.
Transaction process completed.

===== FINAL WALLET DETAILS =====
===== WALLET DETAILS =====
User Name      : Rahul

```

Mobile Number : 9876543210
UPI ID : rahul@upi
Balance : Rs.7500.00

■ Result

The **UPI Payment and Digital Wallet System** was successfully implemented in Java using an **interface, classes, encapsulation, String operations, and custom exception handling**. The program successfully performs wallet creation, money addition, UPI payment, balance checking, UPI validation, amount validation, and insufficient-balance validation using try-catch-finally.